Refinement versus Refresh: Compute Allocation for Closed-Loop Robot Policies

Alba María Téllez Fernández
MAAT Institute
alba.tellez@maatinstitute.com

Abstract

As vision–language–action policies become more computationally demanding, deployment under explicit resource constraints poses an allocation problem: how much inference effort should construct each action chunk, and how long should it execute before prediction is refreshed from a new observation? We study this question by varying generation effort and execution prefix separately, measuring both closed-loop success and complete inference expenditure, and distinguishing prespecified comparisons from later descriptive analyses. We use one frozen SmolVLA checkpoint in synchronous LIBERO simulation as a worked case, with denoising count as its generation control. At a fixed two-action prefix, the prespecified comparison found 139/160 successes with two denoising steps versus 122/160 with five, while using 5.9% less counted arithmetic per control step; 82.1% of generation arithmetic at the default setting was independent of denoising. A later descriptive contrast found 138/160 versus 139/160 successes when executing five rather than two actions per prediction, with 60.7% lower arithmetic per attempt; equivalence was not established. These checkpoint-specific results do not define a general allocation rule. They show why closed-loop behavior, complete inference cost, and the status of each comparison must be considered together.

1 Introduction

Chunked robot policies make two coupled deployment choices. Each policy call constructs an action sequence, the robot executes only a prefix of that sequence, and the policy then predicts again from a newer observation. More computation inside a call may change the predicted actions. Executing a longer prefix reduces the number of calls, but it also delays feedback and commits the robot to later actions from an older prediction. The useful setting therefore depends on both closed-loop behavior and the total cost of prediction over an attempt.

We call adjustable computation spent constructing one action chunk *refinement*, and obtaining a new observation-conditioned prediction *refresh*. The number of actions executed before replanning is the *execution prefix*; it controls how often refresh occurs. We ask: *under an inference budget, how much computation should construct each chunk, and how many actions should the robot execute before the policy refreshes it?*

Our evaluation varies the model-specific refinement setting and execution prefix separately. It counts the arithmetic of each complete policy call under a stated boundary and the resulting expenditure over complete attempts. Discovery tasks are used to choose and freeze comparisons; different held-out tasks are then used to evaluate them. Any comparison added after outcomes were visible is labelled *amended descriptive*, so its interpretation remains separate from the prespecified tests.

This comparison is intended for chunked policies with a controllable generation effort. Depending on the architecture, that control could be integration steps, candidate generation, search, or verification

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Post-Training Adaptation of Robot Foundation Models.

effort. The present study tests only denoising count, and numerical control values should not be compared across architectures. Synchronous execution keeps sequencing and cost boundaries fixed; the study does not determine an asynchronous schedule, physical-robot latency, or a hardware-independent optimum.

We use one frozen SmolVLA checkpoint as a worked case study [1]. Denoising count controls refinement, and the execution prefix controls refresh. We evaluate 12 configurations in 480 discovery attempts, then seven frozen settings in 1,120 held-out attempts on eight tasks excluded from selection. Three denoising comparisons were fixed before the held-out evaluation. Comparisons that change the execution prefix were added later and remain descriptive because they change both observation age and the action positions executed from each chunk.

At a fixed two-action prefix, the prespecified primary comparison gives 139/160 successes with two denoising steps and 122/160 with five, while using less counted arithmetic. At the model's default ten-step setting, 82.1% of generation arithmetic is independent of denoising, which limits the savings available from refinement alone. With two denoising steps, executing five actions per prediction gives 138/160 successes and uses 60.7% less arithmetic per attempt than executing two; the available data do not establish equivalent quality. A simple cost-aware rule also selects the same settings as the fitted response model at all three tested budgets, so fitting has no demonstrated selection advantage here.

The contributions are:

- a formulation that connects model-specific generation effort and execution prefix to prediction rate and complete episode expenditure;
- an evaluation procedure that separates discovery, prespecified held-out comparisons, and later descriptive analyses; and
- a SmolVLA case study that demonstrates the procedure across 12 discovery configurations and seven held-out settings.

2 Related Work

Allocation under a compute budget. Compute-optimal scaling studies ask how a fixed training budget should be divided between model size and data [2]; related embodied work studies scaling in agent pretraining and world models [3]. Including deployment demand can change the preferred allocation [4]. These studies establish the need to state the budget and counting boundary. We instead allocate deployment-time compute within one frozen policy and evaluate the choice by closed-loop task success.

Computation used to generate an action. RoboMonkey spends additional inference compute on action candidates and verification [5]. This is one form of refinement: both proposal and verification cost belong to the policy call. SmolVLA provides the flow-matching policy used in our case study and also studies asynchronous inference [1]. VLA-Perf examines how model architecture, hardware, placement, networking, and asynchronous execution affect VLA inference performance [6]. These works show that arithmetic and elapsed time depend on both the model and its execution system. Our case study holds the checkpoint, hardware, and synchronous execution path fixed so that the effects of refinement and execution prefix can be compared within one system.

Choosing when to refresh. Adaptive Action Chunking uses action entropy to choose how much of a predicted chunk to execute [7]. DVAC uses variation observed during denoising to adapt the execution prefix [8]. ElegantVLA schedules vision–language recomputation, reuse, and action generation during execution [9]. These methods change the schedule within an episode. We map fixed refinement and prefix settings first, separate selection from held-out evaluation, and report both nominal prediction rate and realized episode cost. This design evaluates the axes available to an adaptive method; it does not test whether a fixed schedule is better than adaptation.

3 Allocating Compute Between Generation and Execution

We compare two deployment choices for a frozen chunked policy: the effort used to generate one action chunk and the number of actions executed before requesting a new prediction. The evaluation

procedure measures both nominal compute rate and realized episode expenditure, selects comparisons on discovery tasks, freezes the confirmatory comparisons, and evaluates them on separate tasks. Success and cost are reported together. Later analyses remain explicitly descriptive, and arithmetic savings are kept distinct from latency. The SmolVLA case study instantiates generation effort with its denoising count.

3.1 Compute rate and episode expenditure

Let K be the predicted action-chunk length, $1 \le h \le K$ the execution prefix, and r the architecture-specific generation-effort control. At each policy query, the policy predicts K actions, executes only the first h, discards the remaining $K - h$, and generates a new chunk from a new observation. Thus h controls replanning frequency without changing K. We reserve T_e for the number of actions executed in attempt e, and write $F_{\mathrm{gen}}(r)$ for the measured arithmetic of one generation under a stated counting boundary. The study compares configurations (h, r) within a policy; it does not assume that numerical values of r are comparable across architectures. For complete execution prefixes, the nominal compute rate per simulated control step is

$$b(h, r) = \frac{F_{\mathrm{gen}}(r)}{h}.$$

Let $p_{\mathrm{success}}(h, r)$ denote expected closed-loop success under the specified task distribution. For a candidate set $\mathcal{C}$ and budget ceiling B, the allocation target is

$$(h^\star, r^\star)(B) \in \arg \max_{(h,r)\in\mathcal{C}:\, b(h,r)\le B} p_{\mathrm{success}}(h, r). \tag{1}$$

A configuration need not exhaust the ceiling. This is a deployment compute-rate constraint for a fixed checkpoint, rather than a training-plus-deployment objective.

In the SmolVLA instantiation, $K = 50$ and $r = d$, where d is the denoising count. Figure 1 shows this case-study execution loop. Each policy call predicts a fixed-length chunk, executes only the first h actions, discards the remainder, and refreshes from the resulting observation. The denoising count controls generation effort inside the policy call, while h controls how frequently the policy receives a new observation.

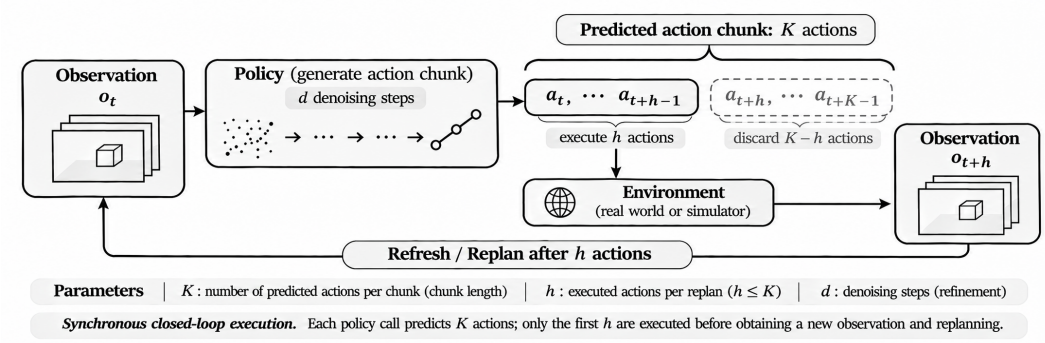


Figure 1: Closed-loop execution in the SmolVLA case study. Each policy call predicts K actions after d denoising steps, executes the first h, discards the remainder, and replans from a new observation. Here $r = d$ and $K = 50$.

We distinguish this rate from total arithmetic per attempted episode. In the case study, generation cost is fixed for a given r. If attempt e executes T_e actions, its generated-chunk expenditure is

$$C_e = \lceil T_e/h \rceil \, F_{\mathrm{gen}}(r). \tag{2}$$

The final generated chunk is charged even if only part of its execution prefix is used. If generation cost varies across calls, as it can for early-exit, sampling, search, or verification procedures, episode expenditure must instead sum the measured cost of every call. All-attempt means include unsuccessful episodes. Different trajectories and termination times can therefore change expenditure at the same nominal rate. FLOPs, host elapsed time, and CUDA-stream time retain separate units and measurement boundaries.

3.2 A fixed-cost floor in the SmolVLA implementation

For the case-study checkpoint and reference input, generation cost separates into a denoising-independent term and a term that grows linearly with d. To rounding, the measured arithmetic is

$$F_{\text{gen}}(d) \simeq 527.108 + 11.457d \quad \text{GFLOPs}$$

at all six calibrated denoising settings. Native generation at $d = 10$ costs 641.675 GFLOPs, of which 82.1% is independent of d. Halving denoising from ten to five saves 8.9% of counted generation work; reducing it to two saves 14.3%. Equivalently, relative to native deployment $(h, d) = (1, 10)$,

$$\frac{b(h,d)}{b_{\text{native}}} \simeq \frac{0.821 + 0.179(d/10)}{h}.$$

Profiling counts matrix products, convolutions, and attention across vision, language prefill, and the action expert, using two FLOPs per multiply-add. Embedding lookups, normalization, activations, elementwise arithmetic, and external preprocessing are excluded. We therefore refer to *dense neural arithmetic*, rather than every scalar operation. Calibration uses one real development observation and fixed tensor shapes; Appendix A.2 gives the procedure.

We prespecify compute-rate ceilings of 0.25, 0.50, and 1.00 times native. Eligibility uses measured per-d values, not the rounded affine expression. For example, $(1, 2)$ costs 0.857 times native rate and is ineligible at half-native budget, whereas $(2, 2)$ costs 0.429 times native. The denoising-independent work alone excludes $h = 1$ at half-native budget and $h \leq 2$ at quarter-native budget. A tighter budget can therefore force a longer execution prefix before any quality-based choice is made. This is an implementation-specific arithmetic floor under the declared counting boundary, not a universal VLA scaling law.

Calibration uses a fixed input shape; a separate language-length audit leaves eligibility unchanged under the original absolute ceilings (Appendix A.5). Reference arithmetic remains distinct from task-shaped arithmetic and elapsed time. For the case-study implementation on a fixed input, reducing d from 10 to 5 lowers measured host generation time by 40.5% while counted arithmetic falls by 8.9% (Appendix A.3); FLOPs are therefore not a latency proxy. The cost model defines feasible schedules; the following experiments ask which feasible expenditures help closed-loop control.

4 SmolVLA Case Study

4.1 Discovery and confirmation design

We evaluate a public SmolVLA checkpoint on LIBERO [10]. The predicted chunk length is fixed at $K = 50$, and the model's native deployment uses $(h, d) = (1, 10)$. We hold the checkpoint, observations, preprocessing, action normalization, control period, and task-specific episode horizons fixed.

The discovery study varies $h \in \{1, 2, 5\}$ and $d \in \{2, 5, 10, 20\}$ on four Spatial tasks. Each task has five initial states and two rollout seeds, giving 40 paired task–state–seed identities. Every identity receives all twelve settings in randomized order, for 480 attempts. Two discovery tasks overlap pre-study pilot work; the other two were chosen before their study outcomes were observed. This partial prior exposure is part of the reason that discovery is used for selection rather than final evaluation.

After discovery, we freeze three denoising comparisons, one at each compute ceiling. Their selected settings and comparators, plus native deployment, produce seven distinct settings. Confirmation evaluates all seven on eight Goal and Object tasks excluded from selection. Each task has ten initial states and two rollout seeds, giving 160 attempts per setting and 1,120 in total. Prior exposure to these tasks during unrelated development is not assessed. No equivalence margin was prespecified, so similar observed success cannot establish equivalent quality. Appendix A.9 records which comparisons were frozen and which were added later as descriptive analyses.

Conditions are paired by initial state and rollout seed. They also share policy noise while their queries occur at the same simulation step. Every denoising setting traverses the same integration interval; changing d changes only its discretization. Internal checks cover initial conditions, generated and

executed actions, and recorded outcomes. They do not provide an independent simulator replay; Appendix A.8 states the validation boundary.

Success gives equal weight to each task; because every task has the same number of states and seeds, this equals the overall success fraction. Descriptive 95% intervals use 20,000 bootstrap draws of states within the fixed tasks while retaining paired seeds and settings. The intervals condition on the four discovery tasks or eight confirmation tasks, and secondary intervals are not adjusted for multiple comparisons. Arithmetic includes every generated chunk, including failed attempts and partly consumed final prefixes. Appendix A.3 reports timing separately.

4.2 Discovery: observed success is not monotonic in refinement

Native $(1, 10)$ succeeds in 25/40 discovery attempts. The highest observed count is 28/40 for $(1, 5)$; $(1, 2)$ and $(2, 2)$ each succeed in 26/40. At $h = 5$, success falls from 21/40 to 14/40 as d increases from 2 to 20, with intermediate counts of 18/40 and 16/40. Appendix Figure 3 shows every configuration. These point estimates do not justify imposing monotonic improvement with denoising. The small paired sample also does not certify equal quality for configurations with similar observed counts.

Selection also depends on how cost is defined. Nominal arithmetic rate differs from realized episode expenditure, and arithmetic savings differ from elapsed time. Replacing the arithmetic ceiling with a host-time ceiling changes which prefixes are feasible. This sensitivity analysis does not alter the frozen arithmetic-based experiment and is reported with the full discovery plots and resource tables in Appendix A.

4.3 Frozen tests and the role of selection

Using discovery data only, the fitted logistic response model selects $(5, 2)$, $(2, 2)$, and $(1, 2)$ at the quarter-, half-, and full-native ceilings. Their frozen comparators are $(5, 5)$, $(2, 5)$, and $(1, 5)$, respectively. The half-native comparison, $(2, 2)$ versus $(2, 5)$, is primary. All selected settings are existing discovery grid points, so confirmation tests task transfer rather than extrapolation to an untested configuration. Each pair changes denoising at a fixed prefix. These are comparisons under common ceilings, not equal-rate tests of how to divide compute between refinement and refresh.

At the primary ceiling, $(2, 2)$ has the highest eligible discovery success. Its comparator $(2, 5)$ ties with $(2, 10)$ for the highest count among the remaining eligible settings and is chosen by the lower-cost tie-break. A simpler rule—use the fewest denoising steps, then the shortest feasible prefix—selects the same three settings as the fitted model. Because this rule was documented after data collection began, the agreement is descriptive. Since both methods select the same settings, confirmation cannot show that fitting improves selection. The model specification, candidate set, and diagnostics remain in Appendix B.

5 Held-Out Results for the SmolVLA Case Study

5.1 Comparisons and evidence status

The held-out evaluation contains 1,120 attempts: seven settings evaluated on the same 160 task–state–seed identities. We write each setting as (h, d), where h is the number of actions executed before replanning and d is the number of denoising steps. Before collecting these outcomes, we froze three comparisons of $d = 2$ against $d = 5$ at fixed $h \in \{1, 2, 5\}$; $(2, 2)$ versus $(2, 5)$ is the primary comparison. Comparisons that change h were added later and are descriptive. Table 1 reports all seven settings, Figure 2 shows their success and compute jointly, and Table 2 states what each comparison supports.

The analysis uses 1,120 canonical attempts. Appendix A.8 documents the exclusion of two repeated records and the retained provenance.

Table 1: Held-out results for the seven SmolVLA settings. Each setting has 160 attempts across eight tasks. The predicted chunk length is fixed at $K = 50$. Nominal rate is counted arithmetic per control step relative to native $(1, 10)$; mean arithmetic per attempt charges every generated chunk.

Allocation		Success		Nominal rate	Mean per attempt	
h	d	Count	(%)	/ native	TFLOPs	Steps
1	2	137/160	85.6	0.857	83.3	151.4
1	5	136/160	85.0	0.911	89.1	152.5
1	10	135/160	84.4	1.000	98.4	153.4
2	2	139/160	86.9	0.429	41.1	149.0
2	5	122/160	76.3	0.455	48.7	166.3
5	2	138/160	86.3	0.171	16.1	144.9
5	5	133/160	83.1	0.182	18.5	156.2

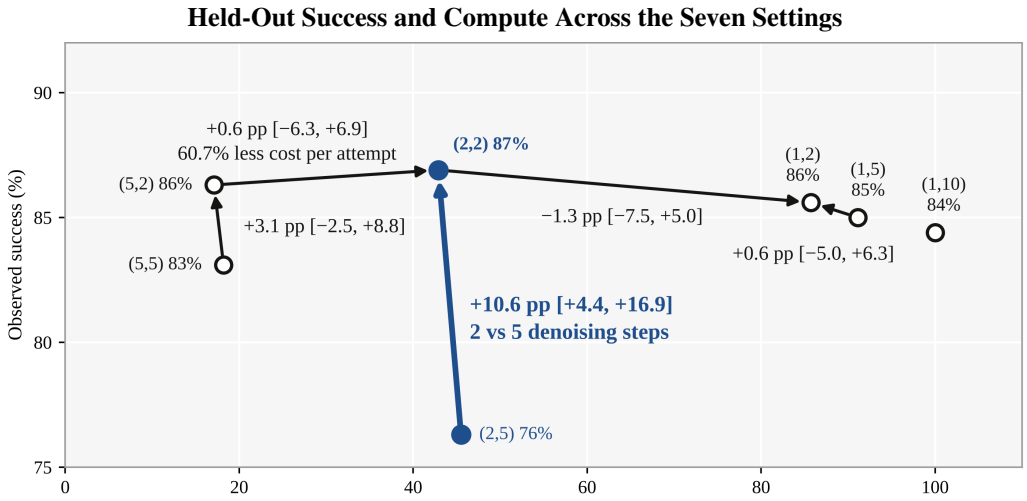


Figure 2: Observed success and nominal arithmetic per control step. Each point is labelled (h, d) and aggregates 160 held-out attempts. Horizontal position is relative to native $(1, 10)$ compute; farther left is cheaper. The success axis begins at 75%. Arrows point from comparator B to setting A and show the A-minus-B success difference with its task-conditional 95% paired interval. Blue marks the prespecified primary comparison. Among the black arrows, those holding h fixed are the other frozen refinement tests, whereas those changing h are amended descriptive prefix contrasts. The 60.7% annotation instead refers to mean arithmetic per attempt for $(5, 2)$ relative to $(2, 2)$; equivalent success is not established.

5.2 Prespecified refinement comparisons

The primary comparison holds the execution prefix fixed at $h = 2$. Reducing denoising from $d = 5$ to $d = 2$ raises observed success from 122/160 to 139/160. The paired difference is $+10.6$ percentage points, with a task-conditional 95% interval of $[+4.4, +16.9]$. It also reduces nominal reference arithmetic per control step by 5.9%, from 292.2 to 275.0 GFLOPs. For this checkpoint and prefix, less refinement therefore improves observed success while reducing counted work.

Mean arithmetic per attempt falls by 15.6%, from 48.7 to 41.1 TFLOPs. This episode-level difference also reflects the changed trajectories: mean episode length is 149.0 control steps for $(2, 2)$ and 166.3 for $(2, 5)$. The primary direction is positive on seven tasks and tied on one, with 24 paired wins and seven losses. Appendix A.7 reports task sensitivity and expenditure intervals. Counted arithmetic does not establish a physical-latency improvement.

The other two prespecified comparisons are smaller and uncertain. At $h = 5$, $d = 2$ gives 138/160 successes and $d = 5$ gives 133/160. At $h = 1$, the counts are 137/160 and 136/160. Both point estimates favor $d = 2$, but both intervals include zero (Figure 2). The evidence therefore supports the primary result at $h = 2$, not a uniform benefit across execution prefixes. The observed $+10.6$-point

effect is also larger than the response model's frozen $+4.0$-point prediction; agreement in direction does not establish calibrated effect-size prediction.

Table 2: Evidence status and permitted interpretation. Effects use A minus B. The table separates prespecified confirmation from amended descriptive analysis.

Comparison	Status	Result	Supported claim
$(2,2)-(2,5)$	Frozen primary	$+10.6$ pp; 95% interval $[+4.4,+16.9]$	At $h=2$, fewer denoising steps improved observed success for this checkpoint.
Other fixed-h refinement tests	Frozen	Positive point estimates; intervals span zero	A uniform benefit is not established for this checkpoint.
$(2,2)-(5,2)$	Amended descriptive	$+0.6$ pp; 95% interval $[-6.3,+6.9]$; $(5,2)$ uses 60.7% less arithmetic per attempt	The case study shows large cost savings; equivalent quality is unresolved.

5.3 Amended descriptive execution-prefix comparisons

These comparisons hold denoising fixed and vary how many actions are executed before replanning. They were added after outcomes were visible and are not confirmatory tests. Appendix A.6 gives their full provenance and the two additional contrasts involving $(1,10)$.

At $d=2$, $(2,2)$ exceeds $(5,2)$ by just one success: 139/160 versus 138/160, a paired difference of $+0.6$ percentage points with 95% interval $[-6.3,+6.9]$. Executing five actions per prediction, $(5,2)$, uses 60% less nominal arithmetic per control step and 60.7% less mean arithmetic per attempt: 16.1 versus 41.1 TFLOPs. The interval permits meaningful success differences in either direction, so the observed counts do not establish equivalent quality. At task level, the $(2,2)-(5,2)$ difference is positive on three tasks, negative on three, and tied on two (Table 7). Thus the nearly equal aggregate counts conceal mixed task-level changes.

Refreshing every action with $(1,2)$ instead of every two actions gives two fewer successes, a difference of -1.3 points with 95% interval $[-7.5,+5.0]$, at 83.3 versus 41.1 TFLOPs per attempt. More frequent refresh has no demonstrated quality benefit in these SmolVLA contrasts. Native-relative comparisons, which also change denoising, are reported separately in Appendix A.6.

Within the case study, the discovery point estimates favor shorter prefixes at $d=2$ (Section 4.2), but their quality advantage is not established on transfer. The task populations differ, so this comparison does not identify a causal task effect. The cost model explains feasibility, but does not determine the quality ranking on new tasks.

6 Discussion and Conclusion

Allocating inference compute in a chunked policy requires three measurements: complete policy-call cost, held-out closed-loop outcomes, and realized expenditure over an attempt. It also requires separating prespecified comparisons from later analyses. Without these elements, a cheap setting can appear useful without behavioral evidence, or a successful setting can be reported without its cost.

The SmolVLA results illustrate the resulting decision boundary. At $h=2$, reducing denoising improved observed success while modestly reducing counted work, but the other frozen prefixes did not establish a uniform benefit. Executing five actions instead of two reduced episode arithmetic much more strongly, while the available data did not establish equivalent quality. Most generation arithmetic was independent of denoising, and the fitted response model selected the same settings as a simple cost-aware rule.

The transferable conclusion is therefore about evaluation rather than a universal setting: generation effort and execution prefix should be studied jointly within each model, using an explicit cost boundary and held-out outcomes. Generation effort may mean integration steps for diffusion or flow policies, or candidate, search, or verification effort for other policies, provided the control and its complete cost are defined in advance. The reported outcomes remain specific to one checkpoint and

simulator stack; broader allocation claims require the same within-model comparison across other checkpoints, policy architectures, tasks, and execution stacks.

Interpretation and limits. The two deployment controls are not isolated causal interventions on prediction quality and observation freshness. Changing denoising changes numerical integration; changing prefix changes both observation age and executed action positions. The mechanism behind the observed primary difference remains open. The study covers one checkpoint and twelve fixed tasks in simulation, with only four discovery task-level validation folds. Intervals are task-conditional, secondary contrasts are unadjusted, and no margin for declaring similar success was prespecified. The refresh-prefix contrasts were added after individual outcomes were visible and remain descriptive. Two steps is the minimum denoising tested; $d = 1$ is untested. The arithmetic boundary excludes several operation classes; language lengths are audited but image shape is fixed. Sampled GPU checks have coverage gaps, and simulated control steps do not establish real-time responsiveness. The recorded outcomes trace to LIBERO's task-goal evaluation, and one native-setting parity episode reproduces the reference evaluator's initial state, action sequence, episode length, and outcome. However, the confirmation actions were not replayed in a separate, version-matched simulator. Outcome validity is therefore strongly supported internally but not fully independently established.

Next steps. A controlled stale-input diagnostic is needed to separate observation freshness from the action-position effects introduced by changing the executed prefix. A predeclared quality-preservation test should evaluate the cheaper $(5, 2)$ setting, and independent outcome replay should test the retained primary trajectories in a separate, version-matched simulator. The same study design can then be applied within other policies and execution stacks: define a controllable generation-effort axis and execution prefix, calibrate complete inference cost, freeze within-policy comparisons, and evaluate success and expenditure jointly. Cross-architecture evidence should compare the within-model allocation results rather than pool unlike control values or counting boundaries. Replication across checkpoints, benchmark families, and asynchronous or physical-robot execution is required to determine when additional generation work is worth paying for and when compute is better spent producing a new chunk from fresher feedback.

References

[1] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. arXiv preprint arXiv:2506.01844v1, 2025. URL https://arxiv.org/abs/2506.01844v1.

[2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Thomas Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karén Simonyan, Erich Elsen, Oriol Vinyals, Jack Rae, and Laurent Sifre. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems*, volume 35, 2022. doi: 10.52202/068431-2176. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/c1e2faff6f588870935f114ebe04a3e5-Abstract.html.

[3] Tim Pearce, Tabish Rashid, David Bignell, Raluca Georgescu, Sam Devlin, and Katja Hofmann. Scaling laws for pre-training agents and world models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48542–48562. PMLR, 2025. URL https://proceedings.mlr.press/v267/pearce25a.html.

[4] Nikhil Sardana, Jacob Portes, Sasha Doubov, and Jonathan Frankle. Beyond chinchilla-optimal: Accounting for inference in language model scaling laws. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 43445–43460. PMLR, 2024. URL https://proceedings.mlr.press/v235/sardana24a.html.

[5] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. In *Proceedings of the 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 3200–3217. PMLR, 2025. URL https://proceedings.mlr.press/v305/kwok25a.html.

[6] Wenqi Jiang, Jason Clemons, Karu Sankaralingam, and Christos Kozyrakis. How fast can I run my VLA? demystifying VLA inference performance with VLA-Perf. arXiv preprint arXiv:2602.18397v1, 2026. URL https://arxiv.org/abs/2602.18397v1.

[7] Yuanchang Liang, Xiaobo Wang, Kai Wang, Shuo Wang, Xiaojiang Peng, Haoyu Chen, David Kim Huat Chua, and Prahlad Vadakkepat. Adaptive action chunking at inference-time for vision-language-action models. arXiv preprint arXiv:2604.04161, 2026. URL https://arxiv.org/abs/2604.04161. CVPR 2026.

[8] Xiangdong Feng, Yuxuan Cheng, Chen Shi, Boyao Han, Yuxuan Yan, Yitong Hong, Zhuotao Tian, and Li Jiang. Denoising tells when to replan: Denoising-variance adaptive chunking for flow-based robot policies. arXiv preprint arXiv:2606.03847v1, 2026. URL https://arxiv.org/abs/2606.03847v1.

[9] Ye Li, Huanan Liu, Kangye Ji, Yuan Meng, Jiajun Fan, Yuansong Wang, Shiyu Qin, Chenglei Wu, Shu-Tao Xia, and Zhi Wang. ElegantVLA: Learning when to think for efficient vision-language-action models. arXiv preprint arXiv:2605.29438v1, 2026. URL https://arxiv.org/abs/2605.29438v1.

[10] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-1939. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/8c3c666820ea055a77726d66fc7d447f-Abstract-Datasets_and_Benchmarks.html.

A Detailed Outcomes and Resource Measurements

This appendix follows the order of the empirical argument: discovery outcomes; generation-cost calibration and timing; sensitivity to the resource coordinate and input shape; complete held-out outcomes and primary expenditure; and finally measurement verification and analysis provenance.

A.1 Discovery outcomes

The checkpoint is `HuggingFaceVLA/smolvla_libero`, revision `6721902bc4d61e50a3bfdb11dfb4cb626f05d102`. Predicted chunk size is 50; native execution uses $h = 1, d = 10$. Discovery uses Spatial tasks 5–8, initial states 10–14, and policy seeds 2000 and 2001. The 20 task–state blocks contain both seeds and all 12 configurations. Discovery bootstrap resampling uses 20,000 draws with analysis seed 20260913.

Table 3: Discovery successes on Spatial tasks 5–8. Each task contributes five states and two seeds per configuration. Rate/native is $F_{\text{gen}}(d)/(hF_{\text{gen}}(10))$, not realized episode cost. The best observed count is 28/40 at $(1, 5)$; no monotonic denoising trend is assumed.

Allocation		Spatial task successes (each /10)					Nominal rate
h	d	Task 5	Task 6	Task 7	Task 8	Total	/ native
1	2	5	7	4	10	26/40	0.857
1	5	5	8	6	9	28/40	0.911
1	10	5	7	6	7	25/40	1.000
1	20	5	6	6	8	25/40	1.179
2	2	5	7	4	10	26/40	0.429
2	5	4	6	4	9	23/40	0.455
2	10	4	6	4	9	23/40	0.500
2	20	4	7	4	9	24/40	0.589
5	2	6	5	4	6	21/40	0.171
5	5	2	5	5	6	18/40	0.182
5	10	2	4	5	5	16/40	0.200
5	20	2	3	4	5	14/40	0.236

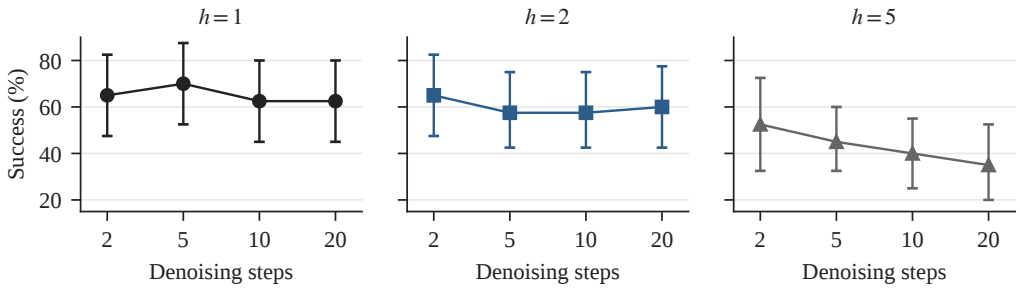


Figure 3: Discovery success by executed prefix h, with 40 attempts per configuration on four Spatial tasks. Connected estimates make the within-prefix response to denoising visible; whiskers are descriptive, task-conditional 95% paired state-bootstrap intervals. The d axes are categorical.

A.2 Generation calibration and discovery resources

Calibration uses one real observation from Spatial task 0, initial state 0, and common explicit noise. For each $d \in \{2, 3, 5, 8, 10, 20\}$, profiling records solver timesteps and dense operation counts, separately from elapsed timing. Each setting receives 20 warm-up generations, followed by three randomized timing blocks of 20 measured calls per setting. The rollout evaluator also performs 20 warm-up generations once per configuration.

Host generation timing brackets prediction and completion synchronization; CUDA events measure the current-stream prediction interval. The outer rollout clock additionally includes observation processing, action handling, simulation/rendering, and record writing. Model loading, environment creation, reset, and warm-up are measured separately. Process sampling occurs approximately every five seconds; the largest recorded gap during discovery is below 5.03 seconds. Display processes remain on the measurement GPU.

Table 4: Discovery resource measurements. Generated-chunk totals use the declared dense-arithmetic boundary and charge partially executed final prefixes. Episode means include unsuccessful attempts. $(2, 2)$ has 26/40 observed successes at 42.9% of native arithmetic rate.

Allocation		Nominal / step	Mean per attempt		Success
h	d	GFLOPs	TFLOPs	Rollout (s)	Count
1	2	550.0	90.6	39.5	26/40
1	5	584.4	91.7	61.8	28/40
1	10	641.7	109.6	111.9	25/40
1	20	756.2	129.1	199.8	25/40
2	2	275.0	45.0	21.8	26/40
2	5	292.2	53.4	38.5	23/40
2	10	320.8	57.8	61.6	23/40
2	20	378.1	67.8	107.8	24/40
5	2	110.0	20.5	12.7	21/40
5	5	116.9	23.8	20.2	18/40
5	10	128.3	27.2	32.0	16/40
5	20	151.2	33.1	55.6	14/40

Calibration requires approximately 291 seconds and discovery 31,245 seconds (8.68 hours) of host elapsed time, including setup and bookkeeping. This search expenditure is additional to subsequent deployments. The experiment does not estimate an amortization threshold or a training–deployment cost crossover.

All measurements use an RTX 4060 Ti. Five-second process samples show no unrelated CUDA compute process during calibration, discovery, or the initially scheduled transfer phase; display processes remain. Process samples were not retained for priority completion or the beginning of continuation. Later continuation monitoring also shows no unrelated sampled CUDA compute process, but these coverage gaps make aggregate transfer rollout times descriptive. Host and CUDA-stream generation intervals are measured separately from the outer synchronous rollout interval. Warm-up is excluded from these rollout measurements.

A.3 Fixed-input generation timing

Table 5: Fixed-input full-generation calibration. Each setting has 60 calls in three randomized blocks. Range gives the minimum and maximum host block means, not a confidence interval. Host time falls more steeply with d than counted arithmetic. Host and CUDA columns average separate retained measurements of all 60 calls; the extra decimals distinguish their means, not timing accuracy.

d	GFLOPs	Host (ms)	CUDA (ms)	Host block range (ms)
2	550.0	217.891	217.862	214.5–220.1
3	561.5	269.990	269.970	265.3–272.6
5	584.4	374.321	374.300	366.8–378.1
8	618.8	529.746	529.726	519.9–536.1
10	641.7	629.624	629.604	623.2–642.2
20	756.2	1158.635	1158.608	1133.5–1172.6

The calibration stores full-generation intervals and global operator arithmetic counts, not separate vision, language-prefill, or action-expert timings and arithmetic totals. These records cannot attribute elapsed sensitivity to a particular component, memory traffic, omitted operation classes, synchronization, or software overhead. CUDA-event intervals are elapsed stream measurements, not summed active-kernel durations. No component-level decomposition is inferred.

A.4 Resource-coordinate sensitivity

Arithmetic savings and elapsed savings differ. Mean rollout time is 111.9 seconds per attempt for native deployment, 61.8 seconds for $(1, 5)$, and 21.8 seconds for $(2, 2)$. Their mean generated-chunk expenditures are 109.6, 91.7, and 45.0 TFLOPs, respectively. These are observed all-attempt means under the same task allocation; they include different trajectories and outcomes. Table 4 reports all discovery configurations.

There is also an ordering reversal: $(1, 5)$ uses more arithmetic per attempt than $(2, 20)$ (91.7 versus 67.8 TFLOPs), but less rollout time (61.8 versus 107.8 seconds). These means include different trajectories. On a fixed input, reducing d from 10 to 5 lowers mean generation host time from 629.6 to 374.3 ms (40.5%), despite only 8.9% less counted arithmetic. The calibration therefore also separates arithmetic savings from elapsed savings without trajectory differences. It does not resolve their component-level cause. A FLOP-constrained choice is not necessarily a latency- or throughput-optimal choice.

As a descriptive sensitivity analysis, define calibrated host-time rate $b_T(h, d) = \overline{T}_{\mathrm{gen}}(d)/h$, the amortized fixed-input generation time per simulated control step, and normalize it by native host-time rate. For $(1, 2)$, $(2, 2)$, and $(5, 2)$, the normalized arithmetic rates are 0.857, 0.429, and 0.171, while the normalized host-time rates are 0.346, 0.173, and 0.069. Consequently, $(1, 2)$ is ineligible at a half-native arithmetic ceiling but eligible at a separately normalized half-native host-time ceiling. These resource constraints admit different refresh schedules; the choices do not establish quality optima. The frozen experiment remains arithmetic-based.

A.5 Language-shape transport audit

The profiled batch contains one sample, two $3 \times 360 \times 360$ image tensors, an eight-dimensional state tensor, and language-token and attention-mask tensors of length 20. The checkpoint exposes seven-dimensional actions and generates a 50-action chunk. Evaluation fixes batch size, image preprocessing, state/action dimensions, and chunk size. The tokenizer pads language only to the longest sequence in a batch. Reconstructing the declared task strings with the pinned tokenizer gives 15–17 tokens for discovery and 5–12 for transfer, versus 20 in calibration. We profile every distinct evaluated length for $d \in \{2, 5, 10\}$, preserving all other shapes, and test eligibility against the original absolute ceilings. Table 6 reports the range over evaluated lengths 5–17 against the original length-20 value.

Table 6: Dense generation arithmetic across evaluated language shapes. Budget tests retain the original absolute ceilings; none of the reported transfer configurations changes eligibility.

d	Evaluated lengths GFLOPs (range)	Length 20 GFLOPs	Eligibility changes
2	539.7–547.9	550.0	0
5	573.5–582.2	584.4	0
10	629.8–639.3	641.7	0

In particular, $(2, 10)$ remains within the original half-native ceiling for all evaluated lengths; shorter inputs reduce rather than increase its counted work. The audit changes neither policy execution nor the frozen selections.

A.6 Complete confirmation outcomes

This subsection first reports success for every held-out task and setting, then reports the four execution-prefix comparisons added after collection began. The held-out evaluation uses Goal and Object tasks 5–8, states 20–29, and policy seeds 3000 and 3001. Its 80 task–state blocks each contain both seeds and all seven configurations.

Table 7: Confirmation successes by task and configuration. Each task row has 20 attempts: ten paired initial states, each with two policy seeds. The total aggregates 160 attempts per configuration. The primary $(2, 2)$ setting exceeds $(2, 5)$ on seven tasks and ties on one.

	$h = 1$			$h = 2$ (primary pair)		$h = 5$	
Task	$d = 2$	$d = 5$	$d = 10$	$d = 2$	$d = 5$	$d = 2$	$d = 5$
Goal 5	14	15	14	16	14	18	16
Goal 6	18	17	17	18	15	19	18
Goal 7	20	20	20	19	18	18	18
Goal 8	19	18	18	18	14	18	15
Object 5	20	19	19	20	20	20	20
Object 6	19	18	19	20	17	18	18
Object 7	9	9	10	10	7	7	8
Object 8	18	20	18	18	17	20	20
Total (/160)	137	136	135	139	122	138	133

Table 8: Amended descriptive comparisons using all 160 attempts per setting. These contrasts were documented after collection began and are not originally prespecified or fully outcome-blind. Intervals are task-conditional, unadjusted, and do not test whether two settings preserve quality within a prespecified margin.

Setting A	Setting B	A−B (pp)	95% interval (pp)
$(2, 2)$	$(1, 10)$	$+2.5$	$[-3.8, +8.1]$
$(2, 2)$	$(5, 2)$	$+0.6$	$[-6.3, +6.9]$
$(1, 2)$	$(2, 2)$	-1.3	$[-7.5, +5.0]$
$(1, 2)$	$(1, 10)$	$+1.3$	$[-5.0, +7.5]$

The four amended comparisons are reported in their documented order, using 20,000 paired state-bootstrap resamples and analysis seed 20260913. Table 1 gives their all-attempt resource means. In particular, the $(2, 2)$ minus $(5, 2)$ expenditure difference is $+25.0$ TFLOPs per attempt, with descriptive interval $[+22.8, +27.5]$. This interval describes generated-chunk arithmetic, including the different trajectory lengths; it is not an elapsed-time comparison or evidence of equivalent quality.

A.7 Primary expenditure and runtime

The primary selected configuration $(2, 2)$ has a higher success count than $(2, 5)$ on seven tasks and ties on one (Table 7). There are 24 paired wins and seven losses. In a descriptive leave-one-task-out sensitivity analysis, the equally weighted difference over the remaining seven tasks stays positive, ranging from 9.3 to 12.1 percentage points. This post-outcome analysis does not extend inference to a new task population. The supplementary paired records retain all state and seed identities needed to reproduce the primary interval.

Table 9: Descriptive primary expenditure differences, selected minus comparator at $h = 2$, including all attempts. Intervals use the paired state bootstrap conditional on the eight tasks and were added after outcomes were observed. Timing intervals summarize sampling variation in the observed records; they do not account for unmonitored runtime interference during priority completion.

Mean per attempt	$d = 2$	$d = 5$	Difference	95% interval
Reference TFLOPs	41.1	48.7	-7.6	$[-10.6, -4.5]$
Observed rollout (s)	19.0	34.3	-15.2	$[-17.2, -13.3]$
Control steps	149.0	166.3	-17.3	$[-27.9, -6.5]$

The retained provenance does not characterize numerical precision and attention execution for every model component. Table 10 therefore supports the stated model configuration, but not component- or kernel-level claims.

Table 10: Runtime and evaluation conditions shared by all four confirmation execution phases. Module versions are the loaded versions reported at evaluation time. The supplement retains source digests and distinguishes conflicting installed distribution metadata from these loaded versions.

Setting	Value
Hardware	RTX 4060 Ti, 16 GB; driver 595.84
Runtime	Python 3.12.3; PyTorch 2.7.1+cu126; CUDA 12.6; NumPy 2.2.6; MuJoCo 3.8.1
Evaluation	Batch 1; synchronous simulation; nominal 20 Hz (50 ms control period)
Episode horizons	Spatial and Object: 280 control steps; Goal: 300
Precision controls	Autocast and TF32 disabled; deterministic algorithms enabled
Attention and reuse	Eager language/action attention with float32 scores; within-generation KV cache enabled
Compilation/warm-up	Model compilation disabled; 20 warm-up generations per condition
Evaluator provenance	Source revision and full digest retained in supplement

A.8 Measurement verification

The retained records support a complete internal reconciliation of planned identities, paired states, executed actions, and recorded outcomes. The checks below constrain recording and execution errors; they do not constitute an independent outcome replay in a separate simulator.

Table 11: Internal measurement-verification checklist.

Check	Evidence
Canonical coverage	1,120 confirmation attempts and 171,805 actions; the primary subset remains 320 attempts and 50,451 actions.
Pairing and dispatch	Planned identities, paired initial states and observations, generation counts, denoising counts, and applied actions reconcile.
Excluded records	Two repeated completed attempts are checked and excluded; one interrupted ten-action record has no completed outcome and is excluded.
Outcome path	All 1,122 raw completed records and 172,243 steps reconcile; generated actions match simulator actions and recorded outcome fields agree.
Shared predicate	Reward and success both use LIBERO's task-goal evaluation, traced to BDDL predicates, so their agreement is an internal consistency check.
Native parity	One Spatial episode matches the upstream path in initial state, observation, 70-action sequence, length, and outcome.
Remaining boundary	Independent, version-matched replay and direct terminal-state inspection remain outside this audit.

A.9 Analysis provenance

The three refinement comparisons, identities, and primary analysis were frozen before transfer outcomes. A deadline-driven amendment changed only execution order after 114 attempts per primary setting had been recorded; settings, identities, stopping rules, and inclusion remained fixed. Continuation completed the original identities, and deduplication retained the earliest completed record under the documented phase precedence. The frozen analyses use seed 20260912. Amended prefix contrasts and diagnostics use seed 20260913 and are descriptive because they were added after collection began. No response model was refitted on confirmation outcomes.

Analysis artifact. The accompanying supplement contains the primary episode and action records, planned identities, paired state values, calibrations, frozen predictions, runtime provenance, and analysis code. Its bundled verification procedure reproduces the primary counts, interval, task table, and descriptive expenditure analysis. Documentation specifies the selector features, penalized objective, candidate set, and normalization. Checkpoint weights and simulator assets are not bundled. The continuing repository additionally retains a completed confirmation snapshot and verification code that reproduce the full seven-setting resource table, original and amended comparisons, and task-level counts. The submitted supplement remains separate from these post-submission additions.

B Frozen Selection and Diagnostic Analyses

We compare two logistic response-model families. A compute-only model uses a quadratic in log normalized rate. An allocation model uses $\log h$, $\log(d/10)$, both squared terms, and their interaction. Each family is fitted with ridge penalty 1 or 10. Leave-one-task-out log loss selects the model, which is then refitted on the discovery set. No individual seed repeat is held out while another repeat from the same task remains in training.

The candidate set is $h \in \{1, 2, 3, 5\}$ crossed with $d \in \{2, 3, 5, 8, 10, 20\}$: the 12 discovery configurations and 12 untested configurations. Their calibrated costs determine eligibility; the fitted model selects the highest predicted success under each ceiling. Ties favor lower compute, then lexicographic order.

Table 12: Discovery-based selector choices. The learned choices and comparisons were frozen before transfer; additional selector baselines were documented after collection began. The empirical selector uses the measured grid; other selectors use the declared candidate set. The simple rule exactly matches the learned choices. Selected rate/native is the learned recommendation's arithmetic rate. Frozen comparators and predicted gaps appear in Table 13; the half-native row is primary.

	Selector choice (h, d)				Learned
Ceiling/native	Learned	Simple	Empirical	Compute-only	rate/native
0.25	$(5, 2)$	$(5, 2)$	$(5, 2)$	$(5, 20)$	0.171
0.50 (primary)	$(2, 2)$	$(2, 2)$	$(2, 2)$	$(2, 10)$	0.429
1.00	$(1, 2)$	$(1, 2)$	$(1, 5)$	$(1, 10)$	0.857

Table 13: All original frozen denoising tests: selected minus comparator, holding h fixed within each row. Predicted and observed gaps and intervals are percentage points. The half-native row is primary; secondary intervals are descriptive, task-conditional, and unadjusted.

Ceiling	Setting (h, d)		Success difference (pp)		
/ native	Selected	Comparator	Predicted	Observed	95% interval
0.25	$(5, 2)$	$(5, 5)$	$+6.5$	$+3.1$	$[-2.5, +8.8]$
0.50 (primary)	$(2, 2)$	$(2, 5)$	$+4.0$	$+10.6$	$[+4.4, +16.9]$
1.00	$(1, 2)$	$(1, 5)$	$+2.2$	$+0.6$	$[-5.0, +6.3]$

The simple rule minimizes denoising count, then chooses the shortest feasible prefix. It reproduces every learned recommendation (Table 12); transfer outcomes cannot distinguish these selectors at the tested budgets. The compute-only choices use the original ridge-10 fit. Its quarter- and half-budget choices are absent from the transfer plan, so this evaluation cannot compare those decisions on transfer tasks.

The selected allocation model has log loss 0.7048, compared with 0.7067 for the best compute-only model. This small difference provides no evidence of an added predictive advantage. Under the same mean natural-log binary cross-entropy, constant $p = 0.5$ scores 0.6931 and an intercept-only model fitted separately on each training fold scores 0.7170. The response-model probabilities should therefore not be read as calibrated task-transfer forecasts. The primary test evaluates a recommendation shared with simpler selectors, so it does not establish an additional benefit from model-based ranking.